Chapter 10 – Case Studies (II): Natural Language Problems

IERG4320/IEMS5726 Data Science in Practice | Cheat Sheet

NLP Overview

NLP：让计算机理解和生成自然语言

Applications: sentiment analysis, text classification, chatbots, machine translation, summarization, speech recognition, image captioning.

Why NLP is hard: high-dimensional, sparse, sequential & context-dependent, noisy & unstructured, language-specific.

高维、稀疏、上下文依赖、噪声多、语言差异

LLM: trained on vast text via self-supervised learning; billions–trillions of parameters. Transformer architecture (2017) enabled GPT, BERT, LLaMA, DeepSeek, Gemini.

Semantic Analysis

语义分析：从文本中提取情感/上下文/含义

- **Context**: word meaning study (translation)
- **Emotions**: happy, sad, fear, calm
- **Sentiments**: positive, neutral, negative

Non-ML concepts: hyponyms, meronomy, polysemy, synonyms, antonyms, homonyms.

上下位词、部分整体、一词多义、近义词、反义词、同形异义

Classic ML Approach

经典方法：TF-IDF + 朴素贝叶斯，准确率约0.84

TF-IDF features + Naive Bayes classifier on IMDB (25k train / 25k test). Accuracy ≈ 0.84.

Limitations: cannot handle polysemy/homonyms, no word position, simple model.

Neural Bag-of-Words (NBoW)

NBoW：词嵌入取平均后送入全连接网络分类

Steps: tokenize → embedding lookup → average pooling → FC layers → classification.

分词→查嵌入→平均池化→全连接层→分类

```
class NeuralBagOfWords(nn.Module):
  def __init__(self, vocab_size, embed_dim,
               hidden_dim, pad_idx):
    super().__init__()
    self.embedding = nn.Embedding(
      vocab_size, embed_dim, padding_idx=pad_idx)
    self.fc1 = nn.Linear(embed_dim, hidden_dim)
    self.fc2 = nn.Linear(hidden_dim, hidden_dim)
    self.fc3 = nn.Linear(hidden_dim, 1)
    self.dropout = nn.Dropout(0.3)
    self.relu = nn.ReLU()
    self.sigmoid = nn.Sigmoid()
  def forward(self, x):
    embedded = self.embedding(x)
    bow = torch.mean(embedded, dim=1) # avg pool
    out = self.dropout(self.relu(self.fc1(bow)))
    out = self.dropout(self.relu(self.fc2(out)))
    return self.sigmoid(self.fc3(out)).squeeze(-1)
# Result: Test Acc = 0.8639 (5 epochs)
```

Pros: simple, fast, parallelizable, fixed-size output.

Cons: ignores word order, no context, limited semantics.

优：简单快速；缺：忽略词序和上下文

LSTM for NLP

LSTM：按时间步顺序处理，保留上下文记忆

Sequential processing: each word is a time step.

Takes current embedding + previous hidden state → gates decide what to remember/forget.

每个词按顺序处理，门控机制决定记忆/遗忘

Pros: captures word order, long-range dependencies.

Cons: sequential (slow), fails on very long sequences.

Test Acc ≈ 0.53 with only 5 epochs (needs longer training).

BERT (Bidirectional Encoder)

BERT：双向Transformer编码器，Google 2018

Key ideas: bidirectional context, self-attention, pre-train then fine-tune.

双向上下文 + 自注意力 + 预训练后微调

Input Representation

`input = token_emb + segment_emb + position_emb`

Special tokens: `[CLS]` (classification), `[SEP]` (separator).

Tokenizer: WordPiece (subword-level).

Pre-training Strategies

1. **MLM** (Masked Language Model): mask 15% tokens, predict originals 遮蔽语言模型
2. **NSP** (Next Sentence Prediction): predict if sentence B follows A 下一句预测

BERT Variants

BERT变体：更小更快的版本

- **ALBERT**: lighter, less training time
- **RoBERTa**: Facebook, improved pre-training
- **ELECTRA**: replaced token detection
- **DistilBERT**: compact, faster (66M params)
- **TinyBERT**: similar to DistilBERT

Code: DistilBERT Mini

```
config = {'vocab_size': 30522, 'dim': 256,
  'n_heads': 4, 'n_layers': 4,
  'intermediate_dim': 1024, 'dropout': 0.1,
  'max_length': 256, 'num_labels': 2}

class DistilBertMini(nn.Module):
  def __init__(self, config):
    super().__init__()
    self.embeddings = nn.Embedding(
      config['vocab_size'], config['dim'])
    encoder_layer = nn.TransformerEncoderLayer(
      d_model=config['dim'],
      nhead=config['n_heads'],
      dim_feedforward=config['intermediate_dim'],
      dropout=config['dropout'],
      batch_first=True, norm_first=True)
    self.transformer = nn.TransformerEncoder(
      encoder_layer,
      num_layers=config['n_layers'])
    self.layer_norm = nn.LayerNorm(config['dim'])
    self.dropout = nn.Dropout(config['dropout'])
    self.classifier = nn.Linear(
      config['dim'], config['num_labels'])
  def forward(self, input_ids, attention_mask):
    x = self.embeddings(input_ids)
    key_pad = ~attention_mask.bool()
    x = self.transformer(
      x, src_key_padding_mask=key_pad)
    # Mean pooling over non-padded tokens
    masked = x * attention_mask.unsqueeze(-1)
    pooled = masked.sum(1) / attention_mask\
      .sum(1, keepdim=True)
    pooled = self.dropout(self.layer_norm(pooled))
    return self.classifier(pooled)
# Params: 10.9M (16.6% of DistilBERT)
# Test Acc: 0.8371 (5 epochs)
```

Applications: semantic similarity, text classification, QA, summarization, NER.

Transformer Model

Transformer：完全基于自注意力，无需RNN/CNN

Self-attention: captures interdependencies between all words simultaneously (not sequential).

3 attention types: within encoder, encoder–decoder, within decoder.

三种注意力：编码器内、编解码器间、解码器内

Always use pre-trained models – training from scratch is very expensive (BERT: 8 days on TPU v3; re-training: 2–3 hours).

```
# TinyBERT pre-trained transformer
from transformers import (AutoTokenizer,
  AutoModelForSequenceClassification)
name = "huawei-noah/TinyBERT_General_4L_312D"
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification\
  .from_pretrained(name, num_labels=2)
# Params: 14.4M, Test Acc: 0.8566 (3 epochs)
```

Encoder vs Decoder vs Seq2Seq

编码器(BERT) vs 解码器(GPT) vs 编解码器(T5)

- **Encoder-only** (BERT): bidirectional, strong feature extraction, not for generation
- **Decoder-only** (GPT): autoregressive, great for text generation, unidirectional
- **Seq2Seq** (T5): encoder–decoder, complex I/O, most expensive

GLUE & SQuAD Benchmarks

GLUE：9个NLP任务的标准评估基准

GLUE (9 tasks):

- **CoLA**: grammatical correctness (Matthews Corr.)
- **SST-2**: sentiment classification (Accuracy)
- **MRPC**: paraphrase detection (F1/Acc)
- **QQP**: question duplicate detection (F1/Acc)
- **STS-B**: semantic similarity (Pearson/Spearman)
- **MNLI**: natural language inference (Accuracy)
- **QNLI/RTE/WNLI**: inference tasks (Accuracy)

SQuAD: reading comprehension.
v1.1: extract answer span from paragraph.
v2.0: includes unanswerable questions.
Metrics: Exact Match (EM) + F1 Score.

Text Generation

文本生成：AI生成连贯自然的文字，本质是"下一词预测"

Harder than NLP: must be coherent, different each time, output > input.

Markov Chain (Non-DL)

马尔可夫链：根据前一个词的概率预测下一个词

$P(X_n|X_1, ..., X_{n-1}) = P(X_n|X_{n-1})$
Transition matrix P: row i, col $j = P(X_{n+1} = j|X_n = i)$.
Steady state: $\pi_n = \pi_{n-1}P$ (converges if irreducible & aperiodic).

Markovify: Python package for text generation.
Pros: no DL needed, fast, light on memory.
Cons: mixes styles, no backward prediction, cannot handle polysemy/homonyms.

GPT (Decoder-only Transformer)

GPT：用解码器生成类人文本，也叫大语言模型(LLM)

- **GPT-1** (2018): 117M params, BooksCorpus
- **GPT-2** (2019): 1.5B params, WebText (45TB)
- **GPT-3** (2020): 175B params, few/zero-shot
- **GPT-4** (2023): 1.76T params, multimodal

Why bigger: early layers learn word order/POS; middle layers learn semantics; late layers learn high-level abstractions.

```
from transformers import (AutoTokenizer,
  AutoModelForCausalLM)
model_name = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(
  model_name)
tokenizer.pad_token = tokenizer.eos_token
model = AutoModelForCausalLM.from_pretrained(
  model_name, dtype=torch.float32,
  device_map="cpu")
```

LLaMA (Meta)

LLaMA：Meta开源的大语言模型系列

Free for research & commercial use.
v1 (Feb 2023) → v2 (Jul 2023) → v3 (Apr 2024) → v4 (Apr 2025).
LLaMA 4 Maverick: 17B active × 128 experts ≈ 400B total.
Llama Stack: unified API for inference, RAG, agents, tools, safety, evals.

Model Comparison on IMDB:
Naive Bayes + TF-IDF: 0.84 | NBoW: 0.86
DistilBERT Mini: 0.84 | TinyBERT: 0.86
Takeaway: Transformers win with pre-training + fine-tuning; NBoW is a strong simple baseline.
Transformer靠预训练+微调取胜; NBoW是简单但有效的基线